JavaScript-Einführung für Java-Programmierer

GRG, HOA, WZR

Web- and Mobile Computing

3. März 2026

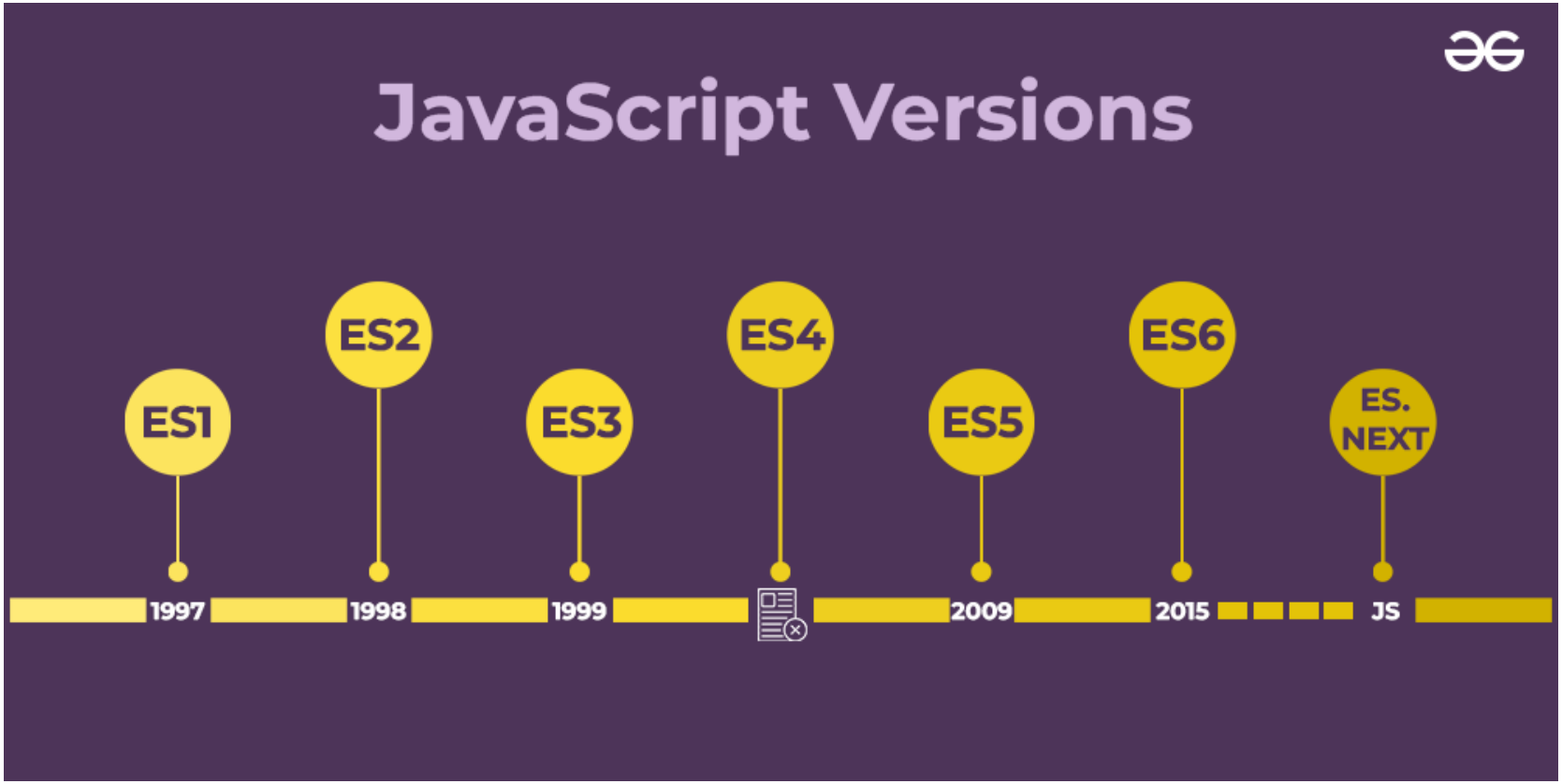
Einleitung

- JavaScript (kurz JS) wird hauptsächlich für client-seitige Programmierung verwendet (Web-Frontend)
- JavaScript wird im Browser *interpretiert*, d.h. der Code wird nicht vorab übersetzt (compiliert), sondern zur Laufzeit
- Syntax von JavaScript ist auf den ersten Blick ähnlich zu Java
- Jedoch: große konzeptuelle und auch syntaktische Unterschiede
- Laufzeitumgebung: Node.js
- Verbreitete Frameworks: `Vue`, `Angular`, `React`, `Svelte`

Entwicklung

- Notwendigkeit einer Programmiersprache für den Browser (Netscape Navigator)
- Enwicklung von Brendan Eich in zehn Tagen (1995)
- Namensgebung in Anlehnung an das damals aufstrebende und mit modernem Internet verbundene Java

Versionen


JavaScript Versions
ES1
ES2
ES3
ES4
ES5
ES6
ES. NEXT
1997
1998
1999
2009
2015
JS

Entwicklung

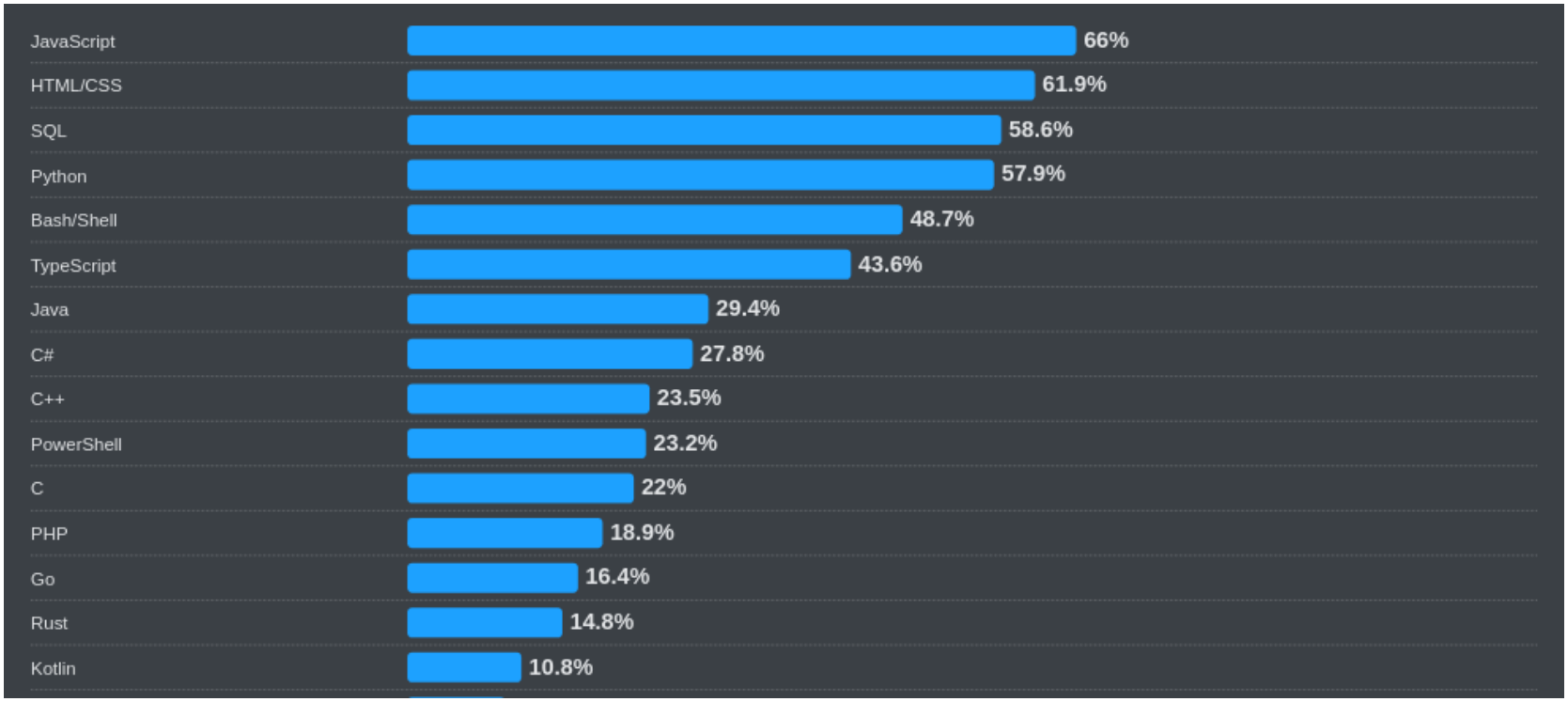


Abbildung: Quelle: Stack Overflow (2025)

Benennung: ECMAScript vs. JavaScript

- **ECMAScript**: Offizieller Standard (ECMA-262) für Skriptsprachen
- **JavaScript**: Implementierung von ECMAScript, entwickelt von Netscape
- Alle modernen JavaScript-Versionen basieren auf ECMAScript
- Im Alltag werden die Begriffe oft synonym verwendet
- Siehe `javascript.tm`

Quellen

- Tutorial `https://www.w3schools.com/js/default.asp`
- Referenz: `https://www.w3schools.com/jsref/default.asp`
- Referenz bei MDN: `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference`

Einbettung in html

Mittels `<script>`-Tags kann JS direkt in html eingebettet werden:

```
...
</p>

<script>
    // Hier her kommt der Javascript-Code

    function foo() {
        alert('Die Funktion foo wurde aufgerufen!');
    }
</script>

<h2>
```

Besser man bindet separate JavaScript-Dateien ein:

```
<script src="Projekt.js" defer></script>
```

Das Attribut `defer` ermöglicht das Einbinden im Header.

node.js

- JavaScript-Laufzeitumgebung auf Basis der V8-Engine (Google)
- Entwickelt von Ryan Dahl (2009)
- Alternative: Deno (Ryan Dahl und Bert Belder)
- Ermöglicht serverseitige (oder lokale) Entwicklung mit JavaScript
- Häufig für Webserver, APIs, Tools und Skripte verwendet
- Riesiges Ökosystem durch npm (Node Package Manager)

npm – Node Package Manager

- Paketmanager für node.js und JavaScript-Projekte
- Ermöglicht Installation, Verwaltung und Veröffentlichung von Paketen (Libraries, Tools)
- Zentrale Registry: https://www.npmjs.com/
- Standardwerkzeug bei node.js-Projekten (`package.json`)
- Unterstützt Skripte und Abhängigkeitsmanagement

Installation

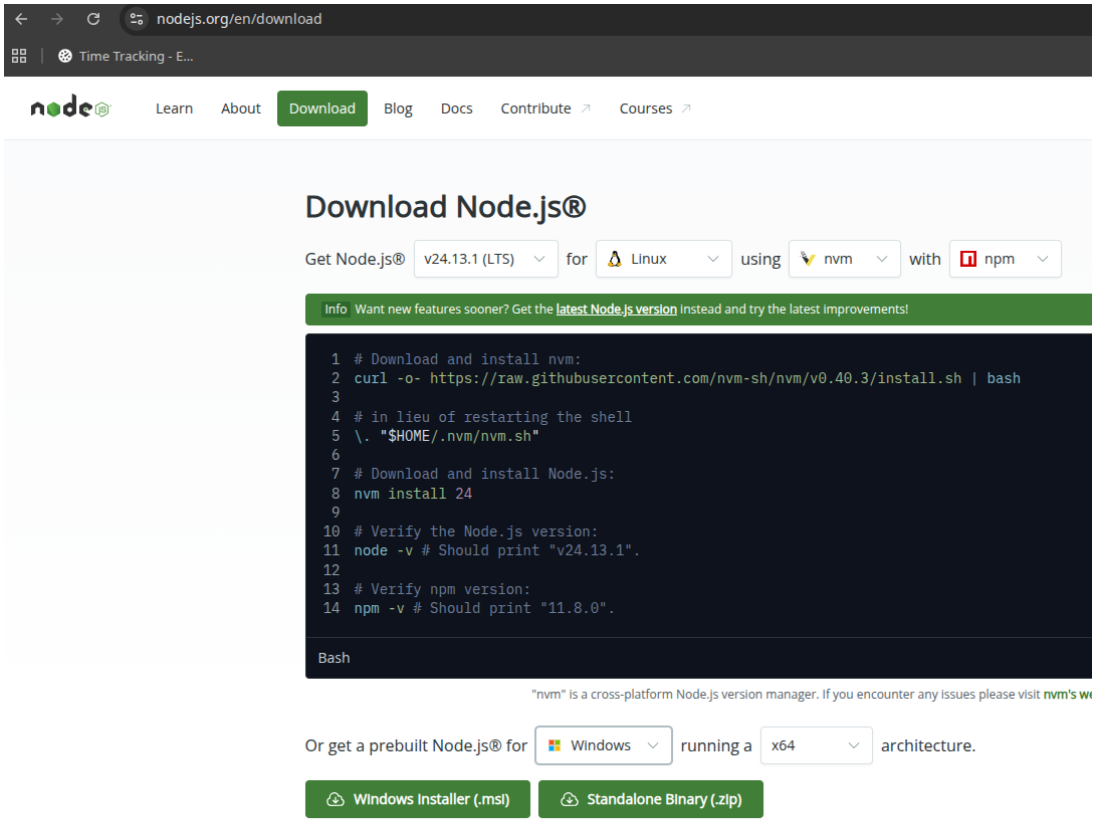


Abbildung: Quelle: `nodejs.org`

Alternativ: `winget install OpenJS.NodeJS`

Node.js und npm: Version und Initialisierung

- Mit den folgenden Befehlen kann die installierte Version angegeben werden von Node.js und npm:

```
node -v
npm -v
```

- Gegebenenfalls unter Windows notwendig:
  `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

- Initialisieren eines neuen npm-Projektes:

```
npm init -y
```

- Dadurch wird eine `package.json` mit Standardwerten erstellt.

Unterschiede zu Java

- Datentypen werden *nicht* angegeben
- Klassen existieren seit ECMAScript 2015 (ES6), jedoch etwas anders: z.B. nur ein Konstruktor
- Objekte unterscheiden sich erheblich zu Java-Objekten
- Ebenso Unterschiede bei Funktionen (vgl. Methoden)

Variablen

- Es werden keine Datentypen angegeben!
- Datentypen sind dynamisch: können sich ändern!
- `const` für unveränderliche Variablen, `let` für veränderliche Variablen, `var` ist **veraltet** (funktionaler Scope)

```
1  function myFunction(x) {
2      if (x > 10) {
3          let a = 10; // Gueltigkeit im Block
4          var b = 20;
5      }
6      // Variable b hier sichtbar
7      // Variable a nicht
8  }
```

Vergleichsoperatoren

- == equal to
- === equal value and equal type
- != not equal
- !== not equal value or not equal type

Datentypen

- String: `let str1 = "Hallo"; let str2 = "World!";`
- Zahlen: `let a = 4; let b = 3.1415;`
  Intern werden alle Zahlen als 64-Bit Fließkommazahl verarbeitet.
  Konversion in Zahl mit `Number("3.14")`.
- Boolean: `let b = false; let c = true;`

Datentypen

Datentyp	Beschreibung
`number`	Repräsentiert Zahlen (Ganzzahlen und Fließkommazahlen doppelter Genauigkeit). Beispiel: 42, 3.14, -Infinity, Infinity, NaN
`bigint`	Für sehr große Ganzzahlen jenseits von `Number.MAX_SAFE_INTEGER`. Beispiel: 1234567890123456789012345678 90n
`string`	Zeichenketten. Beispiel: "Hallo Welt"
`boolean`	Wahrheitswerte: `true` oder `false`
`undefined`	Variable wurde deklariert, aber noch nicht initialisiert
`null`	Repräsentiert die bewusste Abwesenheit eines Wertes
`symbol`	Einzigartige und unveränderliche Werte, oft für Objekt-Eigenschaften verwendet
`object`	Sammlung von Schlüssel-Wert-Paaren oder komplexen Datentypen (Arrays, Funktionen, Objekte)
`function`	Spezielle Art von Objekt, das ausführbaren Code enthält

Funktionen

Funktionen entsprechen den Methoden in Java. Sie müssen aber nicht an Klassen/Objekte gekoppelt sein.

```
1       function berechneSumme(a, b) {
2           return a+b;
3       }
```

Kontrollstrukturen

```
1       if (condition1) {
2           // Code ...
3       } else if (condition2) {
4           // Code...
5       } else {
6           // ...
7       }
8
9       let sum = 0;
10      for (let i=0; i<10; i++) {
11          sum += i;
12      }
```

Objekte

Andere Variablen sind alle vom Typ `Object`.

```
1          let person = {
2              vorname: "Franz",
3              nachname: "Maier",
4              geburtsjahr: 1988,
5              augenfarbe: "blau",
6              ganzerName: function() {
7                  return this.vorname + "␣" + this.nachname;
8              }
9          };
10
11         // Zugriff auf die Objekteigenschaften
12         let x = person.vorname;
13         // oder
14         let y = person["vorname"];
15
16         let z = person.ganzerName();
17
18         console.log(typeof person); // 'object'
```

Klassen

Klassen wurden erst nachträglich zu JS hinzugefügt (seit ECMAScript 2015 / ES6).

```
1   class Auto {
2       constructor(marke, baujahr) {
3           // Parameterpruefung! (hier weggelassen)
4           this.marke = marke;
5           this.baujahr = baujahr;
6       }
7
8       info() {
9           return this.marke + "␣" + this.baujahr;
10      }
11
12      toString() {
13          return this.info();
14      }
15
16  }
17
18  // Objekt erzeugen
19  let auto = new Auto("BMW", 2001);
20  let autoInfo = auto.info();
```

Arrays

```
1   const firmen = ["hp", "IBM", "Lenovo", "Dell"];
2
3   for (let i=0; i<firmen.length; i++) {
4       let x = firmen[i];
5       // mache irgend etwas mit x...
6   }
7
8   // letztes Element entfernen
9   firmen.pop();
10
11  // Hinzufuegen
12  firmen.push("Samsung");
13
14  // leeres Array erzeugen:
15  const laender = [];
```

Ausgabe, Debugging

Statt `System.out.println` in Java verwendet man in JS `console.log()` zur Ausgabe auf die Browser-Konsole.

```
1     ...
2     <script>
3     const auto = new Auto("Mercedes", 1999);
4     const autoInfo = auto.info();
5     console.log("Info␣zum␣Auto:␣" + autoInfo);
6     </script>
7     ...
```

Die Browser-Konsole findet man im Webbroser unter "Developer-Tools" (oder ähnlich).

Ausnahmebehandlung

Mit `throw` können Fehler in Ausnahme- und Fehler-Situationen "geworfen" werden.

```
1     function kinetischeEnergie(masse, geschwindigkeit) {
2         if (masse <= 0) throw "ungueltige␣Masse";
3         if (geschwindigkeit < 0) throw "ungueltige␣geschwindigkeit";
4         return masse * geschwindigkeit * geschwindigkeit / 2;
5     }
6
7     try {
8         kinetischeEnergie(-3, -5);
9     } catch (e) {
10        console.error(e);
11    }
```

Geworfene Fehler werden mit `try .. catch` abgefangen und behandelt.

Aufgaben

Aufgabe 2.1

Befüllen Sie ein Array mit Zahlen und sortieren Sie diese mit einer eigenen Implementierung von Selection-Sort (oder beliebiger anderer Sortieralgorithmus). Der Sortieralgorithmus soll in einer eigenen Funktion umgesetzt sein. Die Vertauschung der Elemente soll ebenfalls in einer eigenen Funktion umgesetzt werden. Geben Sie das sortierte Array auf der Browser-Konsole aus.

Aufgaben

Aufgabe 2.2

Erstellen Sie eine Klasse `Auto` mit den Eigenschaften `Marke`, `Gewicht` und `PS`.[a] Die Funktion `verbrauch()` der Klasse `Auto` soll den Verbrauch näherungsweise berechnen durch PS mal Gewicht, dividiert durch 10000. Die Ausgabe eines Autos auf die Konsole soll so aussehen:

```
Ferrari (1000 kg, 340 PS, Verbrauch: 34 l/100km)
```

Befüllen Sie nun ein Array mit mehreren Autos. Sortieren Sie die Autos nun aufsteigend nach Verbrauch mit einem selbst implementierten Algorithmus (nach den Vorgaben von 2.1). Geben Sie das sortierte Array auf der Browser-Konsole aus.

[a] Die Marke darf nicht leer sein, Gewicht im Bereich 500 bis 3000 kg, PS muss größer als 30 sein.

Type Coercion: typische Beispiele (1/2)

Type Coercion bedeutet **Typumwandlung** in JavaScript.

- **Implizit**: JavaScript wandelt Typen automatisch um (z.B. bei == oder +).
- **Explizit**: Wir wandeln selbst um, z.B. mit `Number(...)` oder `String(...)`.
- Viele "komische" Ergebnisse entstehen durch implizite Umwandlung.

Ausdruck	Ergebnis	Warum passiert das?
`"5" + 3`	`"53"`	+ kann Strings verketten; 3 wird zu `"3"`.
`"5" - 3`	2	- erzwingt Zahlenkonvertierung auf beiden Seiten.
`5 + true`	6	`true` wird zu 1.
`5 + false`	5	`false` wird zu 0.

Type Coercion: typische Beispiele (2/2)

Ausdruck	**Ergebnis**	**Warum passiert das?**
`true == 1`	`true`	Bei == findet implizite Konvertierung statt.
`false == 0`	`true`	`false` wird zu `0` konvertiert.
`"" == 0`	`true`	Leerer String wird zu `0`.
`"" == false`	`true`	Beide Seiten landen bei `0`.
`null == undefined`	`true`	Spezialregel im losen Vergleich.
`null == 0`	`false`	`null` wird hier nicht zu `0`.
`[] == ""`	`true`	Leeres Array wird zu leerem String.
`[] == 0`	`true`	`[]` → `""` → `0`.
`["5"] == 5`	`true`	Array → String `"5"` → Zahl `5`.
`[0] == false`	`true`	`[0]` → `"0"` → `0`.
`"0"== false`	`true`	`"0"` wird zu `0`.
`"\t\n"== 0`	`true`	Whitespace-String wird zu `0`.
`[] + {}`	`"[object Object]"`	Beide Seiten werden als Strings kombiniert.

Type Coercion: Weitere komische Faelle

Ausdruck	Ergebnis	Kommentar
`[] == ![]`	`true`	`![]` ist `false`, danach `[] == false`.
`NaN == NaN`	`false`	Einziger Wert, der nicht gleich sich selbst ist.
`typeof null`	`"object"`	Historischer Bug aus den fruehen JS-Versionen.
`0 < 1 < 2`	`true`	`(0 < 1)` ist `true`; `true` wird zu `1`; dann `1 < 2`.
`2 < 1 < 0`	`false`	`(2 < 1)` ist `false`; `false` wird zu `0`; dann `0 < 0`.